ARTEFACT / TECHNICAL CASE

Empório da Música
Customer Support Agent

Fernando d'Ávila Filho
AI Engineer - Full-Stack Generative AI Applications
Submission overview | November 2026

Project access

Review draft: the implementation is prepared locally. Public repository publication and its URL are pending. This document is not yet ready for submission.

Problem and approach

The store team receives recurring questions about instruments, prices, availability, orders and store rules. Mel is a Python text assistant that retrieves the appropriate evidence before answering, using a friendly and professional persona aligned with the store manual.

The prototype combines local LLM tool calling with deterministic business tools. Product and order questions use structured CSV lookups; policy questions retrieve relevant pages from the original PDF. The interface is a command-line conversation, keeping the focus on correct support behavior.

Architecture

Layer	Implementation
Interaction	CLI with session memory and /reset
Reasoning	Ollama + Qwen3 8B, local inference
Structured lookup	Read-only product, promotion and order tools
Policy retrieval	Lexical page ranking with bilingual keyword hints
Controls	Allowlisted tools, argument validation, bounded loop

Technical decisions

Simple Python orchestration. Three explicit tools make the boundary between generation and factual data easy to inspect. No unrestricted SQL, shell access or write operations are exposed to the model. A five-round budget prevents endless tool loops.

Local inference. Ollama and Qwen3 8B avoid a paid API and keep inference on the host machine. Hardware capacity and model quality remain practical constraints. The model is configurable; the README describes setup and reproducibility limits.

Small-data retrieval. The supplied manual has eight pages and the catalog has 65 products. Lexical PDF retrieval and typed filters are proportionate to this size. Retrieval returns source page numbers; prices and discounts are calculated with Decimal rather than generated by the model.

Session memory. The last four complete turns are retained in memory only. No conversation is persisted to disk by the CLI. A reset clears the conversation.

Important business and privacy boundaries

CSV files are reread on every product lookup, allowing updates to appear in the next response. They remain a supplied snapshot, not a live ERP or carrier integration. Only active products with positive stock are available by default. Only promotions explicitly marked active apply, and PIX discounts never stack with promotional prices.

Order access requires an identity supplied by trusted application code and an ownership check. The CLI has no authentication mechanism, so order requests are redirected to an authenticated store channel. The model cannot establish identity from an order number, email or customer claim.

The manual contains conflicting contact and installment information, an undefined shipping boundary at exactly BRL 500, and overlapping exchange and warranty guidance. Mel is instructed to acknowledge uncertainty and request human confirmation. It must not approve refunds or decide statutory rights.

Order records do not contain the actual receipt date or purchase channel. Online return discussions therefore ask for these facts rather than inferring them from estimated delivery. Complaints, returns and reservations are advisory only; the agent cannot claim they were processed.

Validation and examples

The automated suite passes 21 tests covering catalog filters, pricing, inactive promotions, fresh CSV reads, policy retrieval, order ownership, invalid tool calls, provider errors and loop/memory limits. Scripted responses validate the orchestration protocol and do not establish real-model quality.

Five real-model conversations are included in examples/ and cover catalog search, general information, returns, promotion stacking, privacy and scope.

Running the project

Install Python 3.11+ and Ollama. From the repository root, create a virtual environment, install the package, pull the model and start the CLI. The full README contains troubleshooting and configuration details.

```
python3 -m venv .venv

source .venv/bin/activate

python -m pip install -e .

ollama pull qwen3:8b

ollama serve

python -m emporio.cli

python -m unittest discover -s tests -v

python scripts/evaluate.py
```

Run the CLI in a second terminal while Ollama serves. Skip the serve command if Ollama is already active. The initial model download is approximately 5.2 GB; no paid API key is required.

Limitations and next steps

This is a local CLI prototype. There is no deployed WhatsApp channel, production authentication, real-time inventory, carrier integration or operational handoff. Lexical retrieval can miss paraphrases. Model adherence to grounding and scope instructions is not guaranteed, and read-only tools do not eliminate prompt injection.

Next steps would prioritize a labeled evaluation set and adversarial testing, data validation, pinned runtime/model versions, transactional source APIs, verified customer identity and an auditable handoff queue. A web interface and richer retrieval should follow measured needs rather than precede validation.

AI assistance disclosure

OpenAI Codex assisted with file inspection, implementation, policy ambiguity analysis, regression tests and English documentation. Generated work was checked against supplied data and automated tests. Scripted protocol tests are distinguished from real-model evaluation. The candidate should review and understand all code and assumptions before submission.

Source materials

Artefact technical challenge, supplied operational CSVs, and Empório da Música policy manual v2.1 (June 2025). Original data remain in Portuguese. Technical references: docs.ollama.com/api/chat and docs.ollama.com/capabilities/tool-calling.